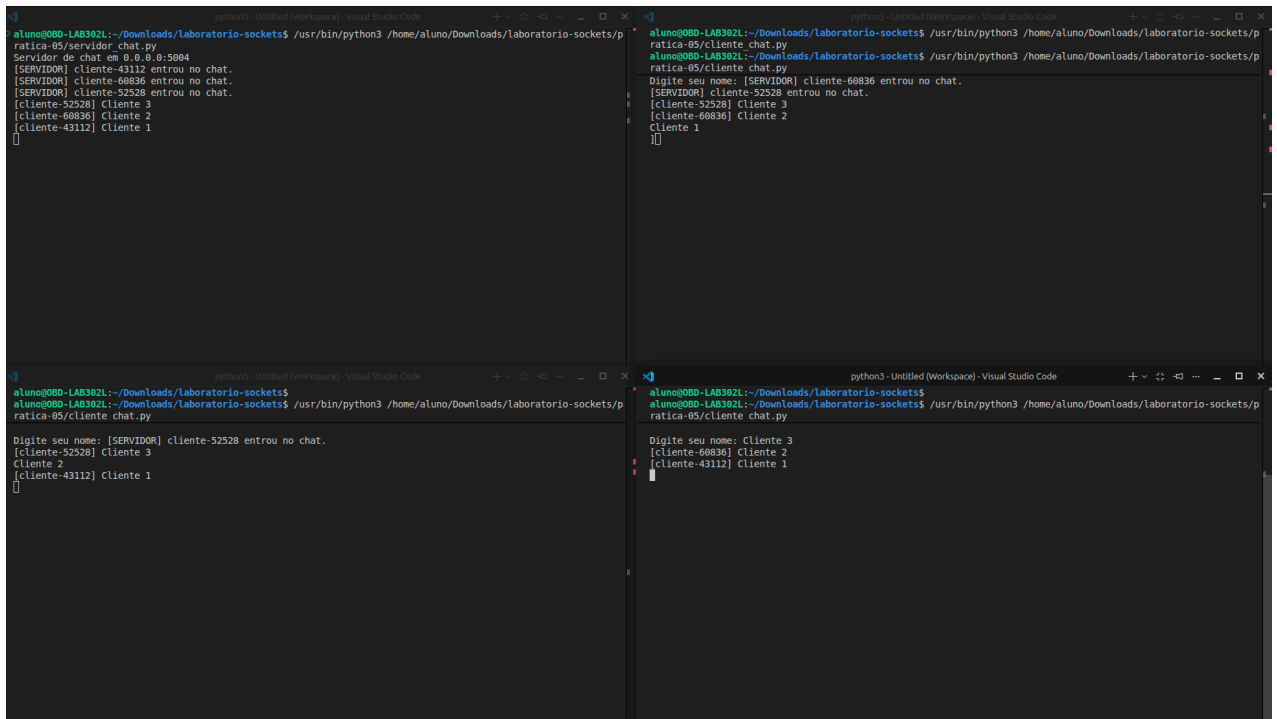


Alunos: Henrique Leão e Rhuan Victor

Prática 5 - Mini-chat TCP em Rede Local



```
aluno080-LAB3021:~/Downloads/laboratorio-sockets$ /usr/bin/python3 /home/aluno/Downloads/laboratorio-sockets/p
ratca-05/servidor_chat.py
Servidor de chat em 0.0.0.0:5004
[SERVIDOR] cliente-43112 entrou no chat.
[SERVIDOR] cliente-60836 entrou no chat.
[SERVIDOR] cliente-52528 entrou no chat.
[cliente-52528] cliente 3
[cliente-60836] cliente 2
[cliente-43112] cliente 1
[]

aluno080-LAB3021:~/Downloads/laboratorio-sockets$ /usr/bin/python3 /home/aluno/Downloads/laboratorio-sockets/p
ratca-05/cliente_chat.py
Digite seu nome: [SERVIDOR] cliente-60836 entrou no chat.
[SERVIDOR] cliente-52528 entrou no chat.
[cliente-52528] cliente 3
[cliente-60836] cliente 2
[cliente 1]
[]

aluno080-LAB3021:~/Downloads/laboratorio-sockets$ /usr/bin/python3 /home/aluno/Downloads/laboratorio-sockets/p
ratca-05/cliente_chat.py
Digite seu nome: [SERVIDOR] cliente-52528 entrou no chat.
[cliente-52528] cliente 3
[cliente 2]
[cliente-43112] cliente 1
[]

aluno080-LAB3021:~/Downloads/laboratorio-sockets$ /usr/bin/python3 /home/aluno/Downloads/laboratorio-sockets/p
ratca-05/cliente_chat.py
Digite seu nome: cliente 3
[cliente-60836] cliente 2
[cliente-43112] cliente 1
[]
```

32. Por que o cliente precisa de uma thread para receber mensagens enquanto o usuário digita?

O método `recv()` do socket em Python é bloqueante. Se não houvesse uma thread separada rodando em segundo plano (daemon), o cliente não conseguiria digitar uma mensagem no `input()` e receber mensagens do servidor simultaneamente. A execução ficaria travada aguardando o usuário digitar algo.

33. Qual é a finalidade do lock no servidor?

O servidor aloca uma thread para cada cliente. O lock serve para garantir a exclusão mútua ao acessar a variável global `clientes` (sua lista). Isso impede condições de corrida (race conditions), garantindo que duas threads não tentem modificar ou ler a lista de conexões no exato mesmo milissegundo, o que poderia corromper os dados ou causar exceções de índice.

34. Que problemas podem ocorrer se dois clientes enviarem mensagens ao mesmo tempo?

Se você não houvesse implementação do lock, os sockets poderiam tentar

escrever dados na rede simultaneamente, misturando pacotes ou derrubando a iteração do laço for.

35. O protocolo do chat está bem definido? O que poderia ser formalizado melhor?

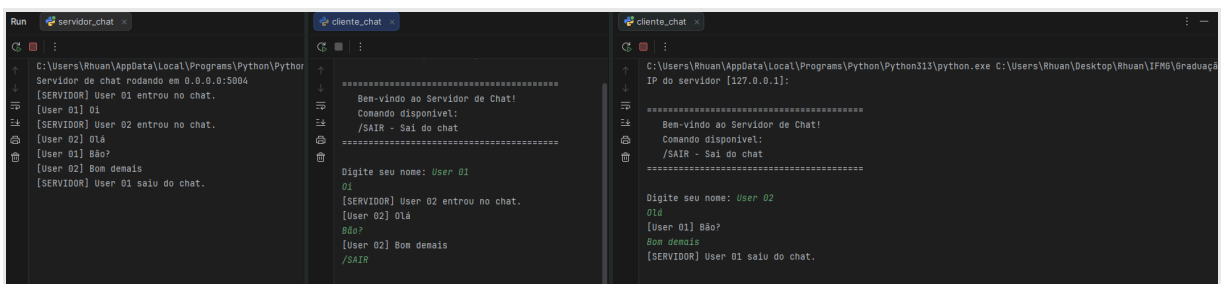
O protocolo atual funciona bem para redes locais e textos curtos, mas é rudimentar. O TCP é um protocolo de fluxo de bytes (byte-stream), e o seu código lê blocos estáticos de 1024 bytes. Faltou formalizar um cabeçalho (como tamanho do payload) ou um delimitador final claro (como quebra de linha `\n` processada via buffer). Se um usuário enviar uma mensagem de 2000 bytes, ela será fatiada, ou se várias mensagens pequenas chegarem rápido demais, elas podem ser aglutinadas no mesmo recv. Mudar para um formato estruturado como JSON resolveria isso.

36. Quais limitações esse chat teria se fosse usado por muitas pessoas?

O modelo de concorrência escolhido é o Thread-per-Connection (uma thread por cliente). Se mil pessoas se conectarem, o servidor teria mil threads ativas, consumindo muita memória RAM e sobrecarregando o sistema operacional com trocas de contexto (context switching).

Implementação de uma variação

Foi implementada uma mensagem de boas-vindas com instruções de uso. A lógica foi feita no lado do servidor, logo no início da função `atender_cliente(conexao, endereco)`. Antes de aguardar a definição do nome de usuário pelo `recv()`, o servidor despacha um bloco de texto via `sendall()` contendo o cabeçalho do chat e o comando disponível. Isso garante que qualquer pessoa que se conecte a partir de um terminal saiba imediatamente como interagir com o sistema.



```
Run
C:\Users\Rhuan\AppData\Local\Programs\Python\Python313\python.exe C:\Users\Rhuan\Desktop\Rhuan\IPMG\Graduaça
servidor_chat
Servidor de chat rodando em 0.0.0.0:5004
[SERVIDOR] User 01 entrou no chat.
[User 01] Oi
[SERVIDOR] User 02 entrou no chat.
[User 02] Olá
[User 01] Bão?
[User 02] Bom demais
[SERVIDOR] User 01 saiu do chat.
Process finished with exit code 0

cliente_chat
=====
 Bem-vindo ao Servidor de Chat!
 Comando disponível:
 /SAIR - Sai do chat
=====
Digite seu nome: User 01
Oi
[SERVIDOR] User 02 entrou no chat.
[User 02] Olá
Bão?
[User 02] Bom demais
/SAIR

cliente_chat
C:\Users\Rhuan\AppData\Local\Programs\Python\Python313\python.exe C:\Users\Rhuan\Desktop\Rhuan\IPMG\Graduaça
IP do servidor [127.0.0.1]:
=====
 Bem-vindo ao Servidor de Chat!
 Comando disponível:
 /SAIR - Sai do chat
=====
Digite seu nome: User 02
Olá
[User 01] Bão?
Bom demais
[SERVIDOR] User 01 saiu do chat.
```

Descrição do protocolo de comandos do chat

O protocolo de comunicação da aplicação foi estruturado sobre a camada de transporte TCP (porta 5004), operando no modelo cliente-servidor. A troca de mensagens ocorre em texto plano, utilizando a codificação UTF-8, com o caractere de quebra de linha (\n) atuando como delimitador visual no terminal.

O fluxo de comunicação e os comandos obedecem às seguintes regras:

- **Aperto de mão na aplicação (Handshake):** Assim que a conexão TCP é estabelecida, o servidor envia imediatamente uma mensagem de boas-vindas com as instruções. O cliente, em seguida, deve enviar uma string contendo o nome do usuário.
- **Identificação Padrão:** Se o cliente enviar uma string vazia no momento do registro do nome, o servidor atribui um nome genérico no formato cliente-<porta_de_origem>.
- **Comando de Saída (/SAIR):** É o único comando de controle implementado pelo cliente. Ao ser digitado, o laço de repetição de envio é interrompido no cliente, e o socket é fechado. O servidor detecta o encerramento, remove o cliente da lista ativa e notifica os demais usuários.
- **Mensagens de Broadcast:** Qualquer entrada de texto diferente de /SAIR enviada por um cliente é interpretada pelo servidor como uma mensagem pública. O servidor anexa o nome do remetente no formato [Nome] mensagem\n e encaminha a string resultante para todos os outros sockets conectados.